PRESENTACION

PROGRAMACION 2

ARQUITECTURA DEL PROYECTO
"TE ATIENDO"

Estudiantes

WALDIN CEBALLOS 2025-1112

ROBERT FRANCISCO LECLERC 2025-1390

Profesor

FRANCIS RAMIREZ

Fecha

2/01/2026

Documento de Arquitectura de Software (SAD):

Plataforma de Reservas y Órdenes de Restaurantes

Integrantes

Robert Francisco Leclerc Dionisio

Vista Lógica

Vista de Procesos

Vista de Desarrollo

Vista de Despliegue

Vistas de Caso de Usos

Conclusiones

Waldin Ceballos Ramirez

Introduccion

Vista General del Sistema

Componentes del Sistema

Principios y Decisiones Arquitectónicas

Modulos del sistema

Decisiones Arquitectónicas

Riesgos Técnicos

Introducción

El presente Documento de Arquitectura de Software (SAD) describe la estructura, componentes y decisiones técnicas fundamentales para el desarrollo de la plataforma **"Te Atiendo"**. Este sistema nace como una solución integral para optimizar la interacción entre comensales y establecimientos gastronómicos, centralizando procesos que tradicionalmente se gestionan de forma fragmentada.

El propósito de este documento es servir de guía técnica para el equipo de desarrollo, asegurando que la implementación sea coherente con los requerimientos de reserva, pedidos anticipados y auditoría definidos previamente. A través del modelo de vistas 4+1, se detallarán los aspectos lógicos, físicos y de proceso que garantizan un sistema robusto, seguro y escalable.

Alcance del Proyecto

El proyecto **"Te Atiendo"** abarca el diseño y desarrollo de un ecosistema digital compuesto por tres frentes operativos:

- **Para los Clientes:** Una interfaz intuitiva que permite la búsqueda de restaurantes, gestión de reservas de mesas en tiempo real, creación de pedidos anticipados (órdenes) y un sistema de reseñas basado en experiencias verificadas.
- **Para los Restaurantes:** Un panel de gestión para administrar la disponibilidad de mesas, actualizar menús, procesar órdenes entrantes y visualizar el historial de transacciones.
- **Para la Administración:** Una consola de supervisión general para la moderación de contenido (reseñas), gestión de usuarios y acceso a registros de auditoría para garantizar la transparencia del sistema.

Fuera de Alcance:

- El sistema no gestionará servicios de entrega a domicilio (delivery).
- No se incluye la gestión de inventarios de almacén o proveedores de los restaurantes.
- El procesamiento de pagos se limitará a la simulación y validación de transacciones mediante una pasarela externa integrada, sin manejar directamente datos bancarios sensibles en servidores propios.

Visión General de la Arquitectura

Objetivo

El objetivo de la arquitectura del sistema **"Te Atiendo"** es definir una estructura tecnológica organizada, escalable y mantenible que permita soportar de manera eficiente los procesos de reserva de mesas, gestión de órdenes anticipadas y control administrativo de restaurantes afiliados.

La arquitectura tiene como objetivo asegurar que el sistema sea capaz de gestionar distintos perfiles de usuario (Cliente, Restaurante y Administrador), procesar pagos de forma segura, validar disponibilidad en tiempo real y mantener un registro de auditoría exhaustivo para garantizar la transparencia en las operaciones gastronómicas.

Modelo Arquitectónico

Se ha seleccionado una **Arquitectura por Capas (N-Tier)** combinada con un enfoque de **Servicios Web (Web API)** para el desarrollo de la plataforma.

Este modelo permite dividir el sistema en niveles de responsabilidad claros:

- **Capa de Presentación:** Implementada en **Blazor** para los clientes y administradores, permitiendo una interfaz reactiva que se comunica vía HTTPS con el núcleo del sistema.
- **Capa de Lógica de Negocio (Backend):** Centralizada en **ASP.NET Core**, donde se procesan las reglas de validación de reservas y órdenes.
- **Capa de Datos:** Gestionada por **SQL Server** para el almacenamiento persistente y seguro de la información.

La selección de este modelo se basa en las siguientes ventajas:

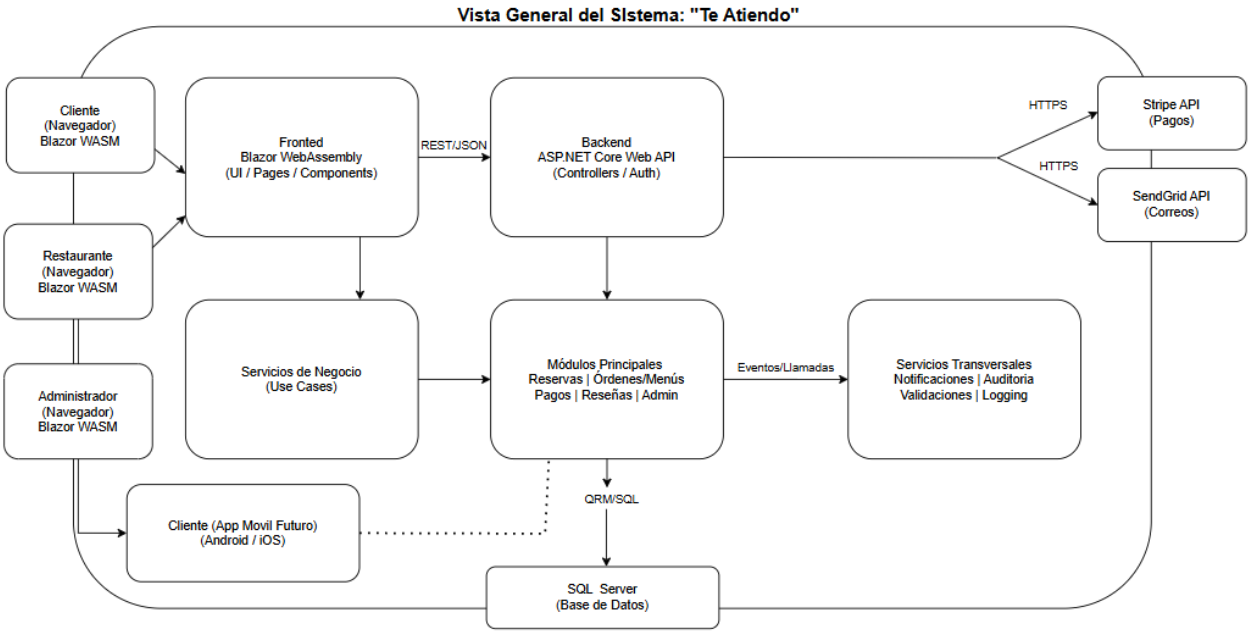
- **Separación de responsabilidades:** Divide la interfaz de usuario del procesamiento lógico y el almacenamiento de datos.
- **Centralización de reglas:** Garantiza que validaciones críticas (como la disponibilidad de mesas o autenticidad de reseñas) se ejecuten en un solo punto.
- **Escalabilidad:** Permitira en un futuro integrar APIs externas como **Stripe** para pagos y **SendGrid** para notificaciones sin comprometer la estabilidad del sistema.
- **Trazabilidad:** Facilita el cumplimiento del requisito de registrar cada acción importante para fines de auditoría.

Principios Arquitectónicos

- **Centralización del Dominio:** Las reglas de negocio (validación de disponibilidad, estados de órdenes, autorización de pagos y políticas de cancelación) residen exclusivamente en el Backend y no en las interfaces de usuario para evitar inconsistencias.

- **Separación de Responsabilidades (SoC):**
    - **Presentación:** Responsable de la interacción con el usuario en el navegador mediante componentes de Blazor.
    - **Aplicación/API:** Encargada de orquestar los casos de uso como el registro de usuarios, creación de reservas y procesamiento de pedidos.
    - **Persistencia / Infraestructura:** Gestiona el almacenamiento en SQL Server y la comunicación con servicios externos de correo y pagos.
- **Trazabilidad Transversal:** Toda operación relevante (confirmación de pago, cambio de estado de reserva o envío de notificación) es registrada indicando el actor, la operación y la marca de tiempo.
- **Consistencia en Operaciones Críticas:** Las transacciones relacionadas con la reserva de mesas y el pago de órdenes se ejecutan de forma atómica para evitar duplicidad de cupos o estados financieros incorrectos.
- **Evolución y Extensibilidad:** El desacoplamiento entre el Frontend y el Backend permite que en el futuro se incorporen aplicaciones móviles nativas o integraciones con sistemas POS sin necesidad de reescribir la lógica central.

3. Vista General del Sistema



Descripción de alto nivel de la interacción de módulos

El sistema opera bajo un flujo de comunicación bidireccional y seguro, diseñado para garantizar que las reglas de negocio se apliquen de forma consistente en cada transacción.

Básicamente, el sistema arranca con el **Frontend en Blazor**, que es la parte visual donde los clientes y dueños de webaurantes entran desde su navegador. Cada vez que alguien hace algo, como buscar una mesa o pedir comida, la web le manda un mensaje seguro por **HTTPS** al **Backend**.

El **Backend** es el que manda y funciona así:

- Recibe la petición y lo primero que hace es chequear quién eres para dejarte pasar según las reglas de acceso.
- Luego, mueve los hilos en los **Módulos de Negocio** para validar si hay mesas libres o si el menú está correcto antes de confirmar nada.
- Si toda cuadra, guarda la información en la **Base de Datos SQL Server** para que nada se pierda y quede registrado.

Lo más importante es cómo se conecta con el exterior. Si hay un pago, el Backend habla con la **API de Stripe** para autorizarlo. En cuanto el pago sale bien, le da la orden a la **API de SendGrid** para que dispare el correo de confirmación al cliente de una vez. Al mismo tiempo, todo se anota en un registro de **Auditoría** para que el administrador pueda revisar qué pasó en el sistema.

4. Vistas Arquitectónicas (Modelo 4+1)

Casos de uso Casos de Uso

Describe las funcionalidades del sistema desde la perspectiva del usuario y los actores involucrados.

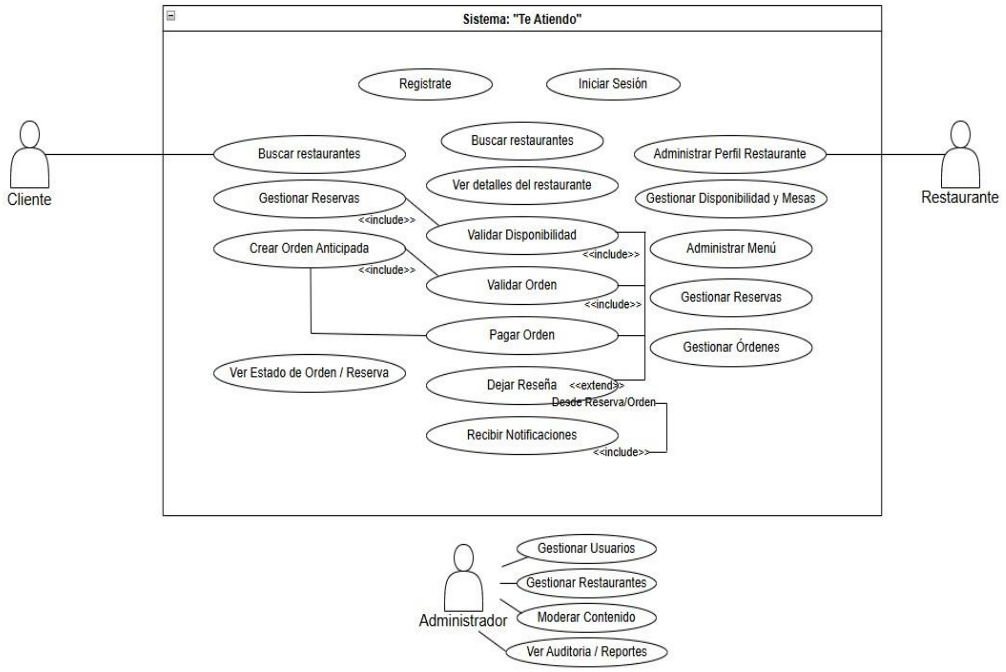
Actores principales:

- **Cliente:** Usuario que busca, reserva y consume en los restaurantes.
- **Restaurante:** Personal del establecimiento que gestiona mesas, menús y pedidos.
- **Administrador:** Personal encargado de la supervisión global, moderación y auditoría del sistema.

Casos de uso principales:

- **Gestionar Reservas:** Permite al cliente asegurar un espacio y al restaurante confirmar la disponibilidad.
- **Crear Órdenes Anticipadas:** Facilita la selección y preparación previa de platos para optimizar el servicio.
- **Validar Disponibilidad:** Proceso automático obligatorio para evitar conflictos de horarios o sobreventa.
- **Pagar Orden:** Integración con servicios financieros para procesar transacciones de forma segura.
- **Notificar Usuario:** Envío automático de confirmaciones y estados de órdenes por correo electrónico.
- **Consultar Auditoría y Reportes:** Registro detallado de operaciones para fines de control administrativo.

Diagrama de Casos de Uso General

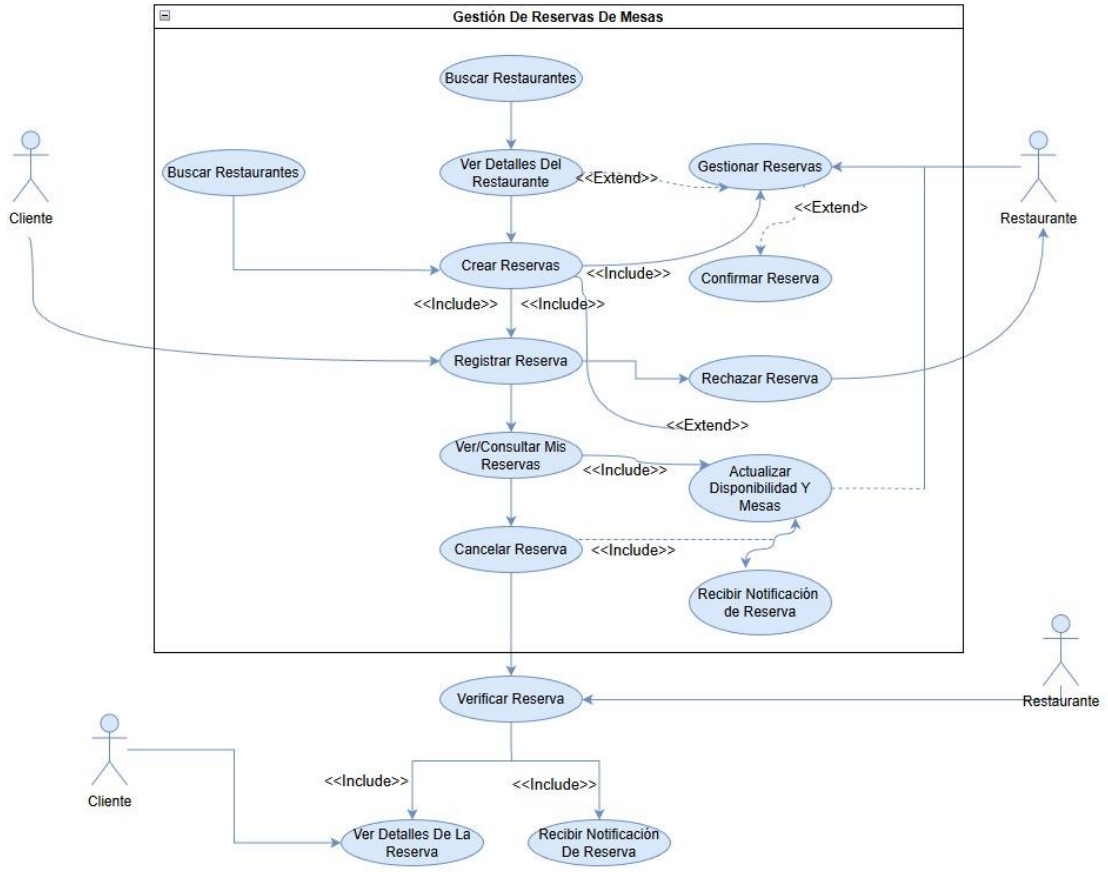


Básicamente, el diagrama muestra a las tres personas que van a usar "Te Atiendo": el Cliente, el dueño del Restaurante y el Administrador.

El flujo es sencillo: el cliente entra para buscar un sitio, hace su reserva o pide comida por adelantado, y al final paga y deja su opinión. Por otro lado, el restaurante se encarga de organizar sus mesas, actualizar el menú y aceptar o rechazar los pedidos que le van llegando.

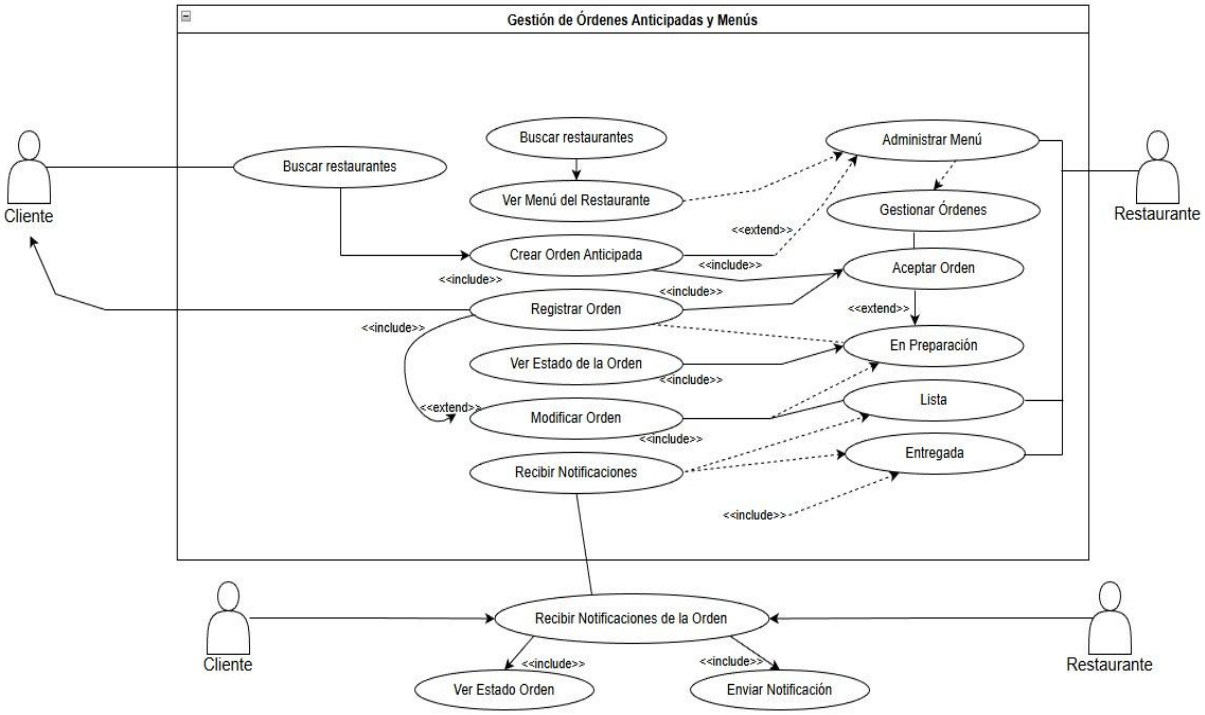
Lo más importante que se ve ahí es que el sistema no deja que nadie reserve o pida comida si antes no se valida que haya disponibilidad o que la orden esté bien hecha; eso el programa lo hace obligado. Además, cada vez que pasa algo importante, el sistema le manda una notificación a la gente para que estén al tanto. Ya al final, el administrador es el que se mete para chequear que todo esté en orden, ver los reportes y manejar a los usuarios

Gestión de Reservas



La Gestión de Reservas de Mesas se define como el proceso integral mediante el cual el sistema "Te Atiendo" coordina la interacción entre el cliente y el restaurante para asegurar un espacio físico en un horario determinado. Este flujo inicia con la solicitud del cliente tras consultar la información del local, lo que dispara automáticamente una validación de disponibilidad que actualiza el inventario de mesas en tiempo real para evitar duplicidad de horarios. El proceso se completa cuando el restaurante ejerce el control operativo al confirmar o rechazar la petición, activando de inmediato el servicio de notificaciones que mantiene a ambas partes informadas sobre el estado final y los detalles de la verificación de la cita.

Ordenes y Menus

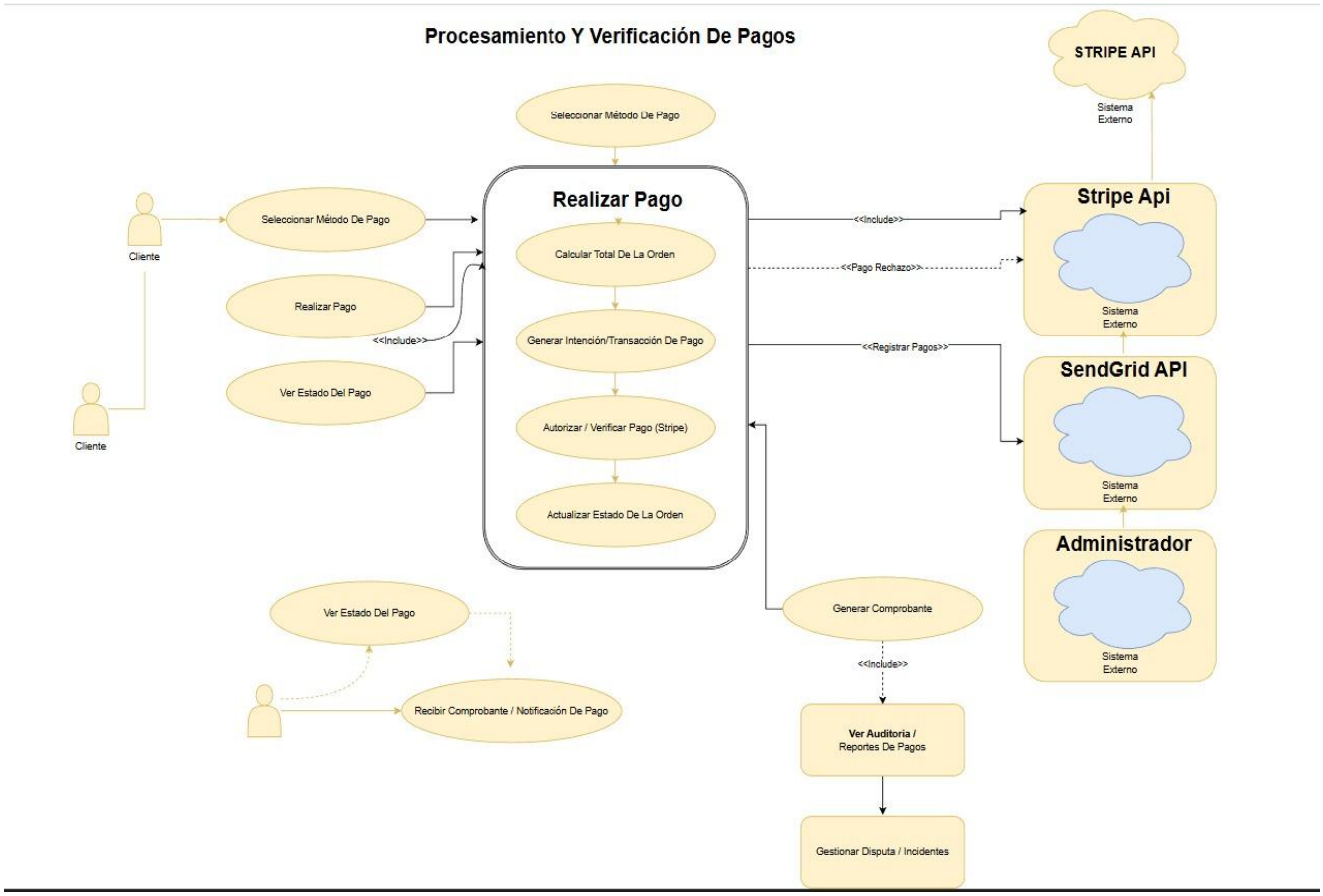


La Gestión de Órdenes Anticipadas y Menús se define como el proceso operativo y logístico del sistema "Te Atiendo" que permite la preselección de alimentos por parte del cliente, facilitando una experiencia gastronómica sin esperas y una mejor planificación para la cocina del restaurante. El flujo comienza cuando el cliente explora el catálogo de platos, lo que desencadena una validación automática de la orden para confirmar que cada elemento seleccionado cumple con las reglas del menú y la disponibilidad del local.

Una vez registrada la solicitud, el restaurante asume el control del flujo de trabajo, pasando por diferentes estados operativos que incluyen la aceptación, la preparación, el aviso de que la orden está lista y, finalmente, la entrega. Todo este ciclo de vida está soportado por un motor de eventos que genera notificaciones constantes hacia el cliente, permitiéndole verificar el estado de su pedido en tiempo real y garantizando que tanto el usuario como el establecimiento

estén perfectamente sincronizados desde el momento de la creación hasta el cierre de la transacción.

Procesamiento y verificación de pagos



La Procesamiento y Verificación de Pagos se define como el flujo crítico de seguridad y finanzas del sistema "Te Atiendo", encargado de gestionar las transacciones económicas entre el cliente y la plataforma de forma transparente y automatizada. Este proceso comienza cuando el cliente selecciona su método de pago y el sistema calcula el total de la orden, generando una intención de pago que debe ser autorizada y verificada en tiempo real a través de la integración con la Stripe API.

Vista Lógica

Describe la estructura interna del sistema, incluyendo clases, servicios, módulos y componentes funcionales.

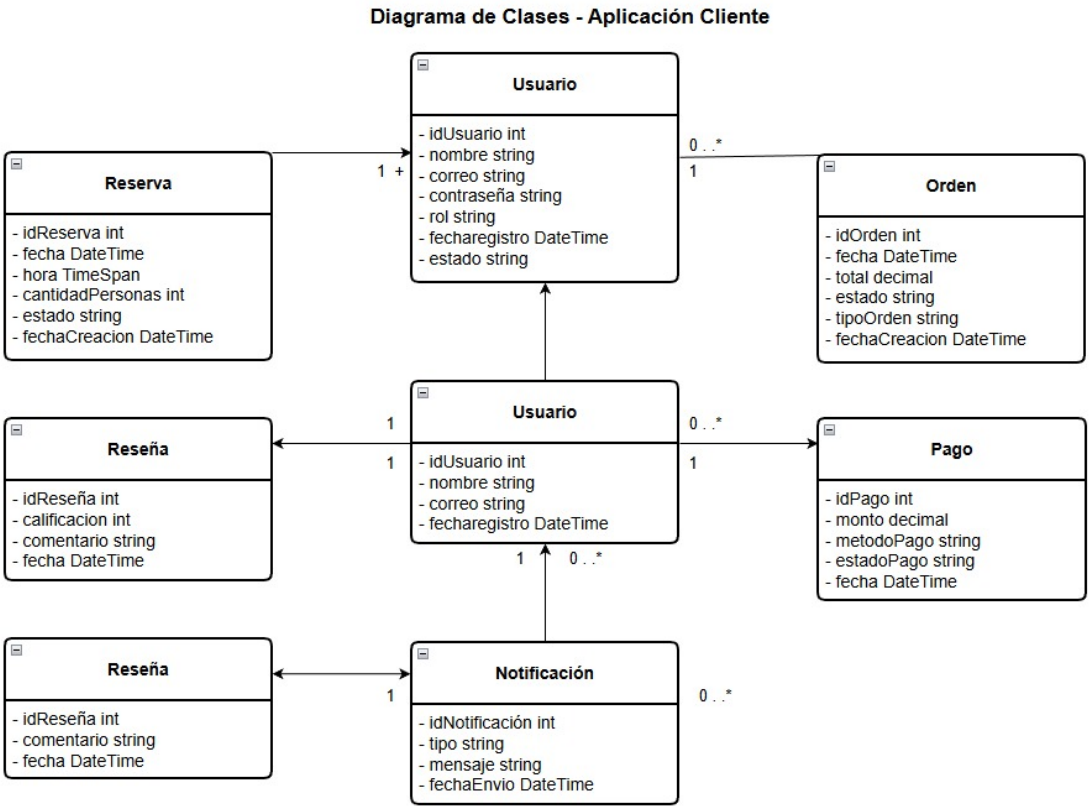
El sistema se organiza en una arquitectura en capas:

- **Presentación**: Implementada con componentes de **Blazor** donde residen las interfaces para el Cliente, Restaurante y Administrador.
- **Aplicación**: Capa encargada de orquestar los procesos mediante la **Web API**, manejando la lógica de las Reservas, Órdenes y Pagos.
- **Dominio**: Centraliza las reglas de negocio representadas en el **Diagrama de Clases**, donde se definen las entidades de Usuario, Mesa, Plato y Reserva, asegurando que los datos sigan las reglas del restaurante.
- **Persistencia / Infraestructura**: Gestiona el almacenamiento de la información en **SQL Server** y la comunicación con las APIs de **Stripe** y **SendGrid**.

El dominio centraliza las reglas de negocio relacionadas con:

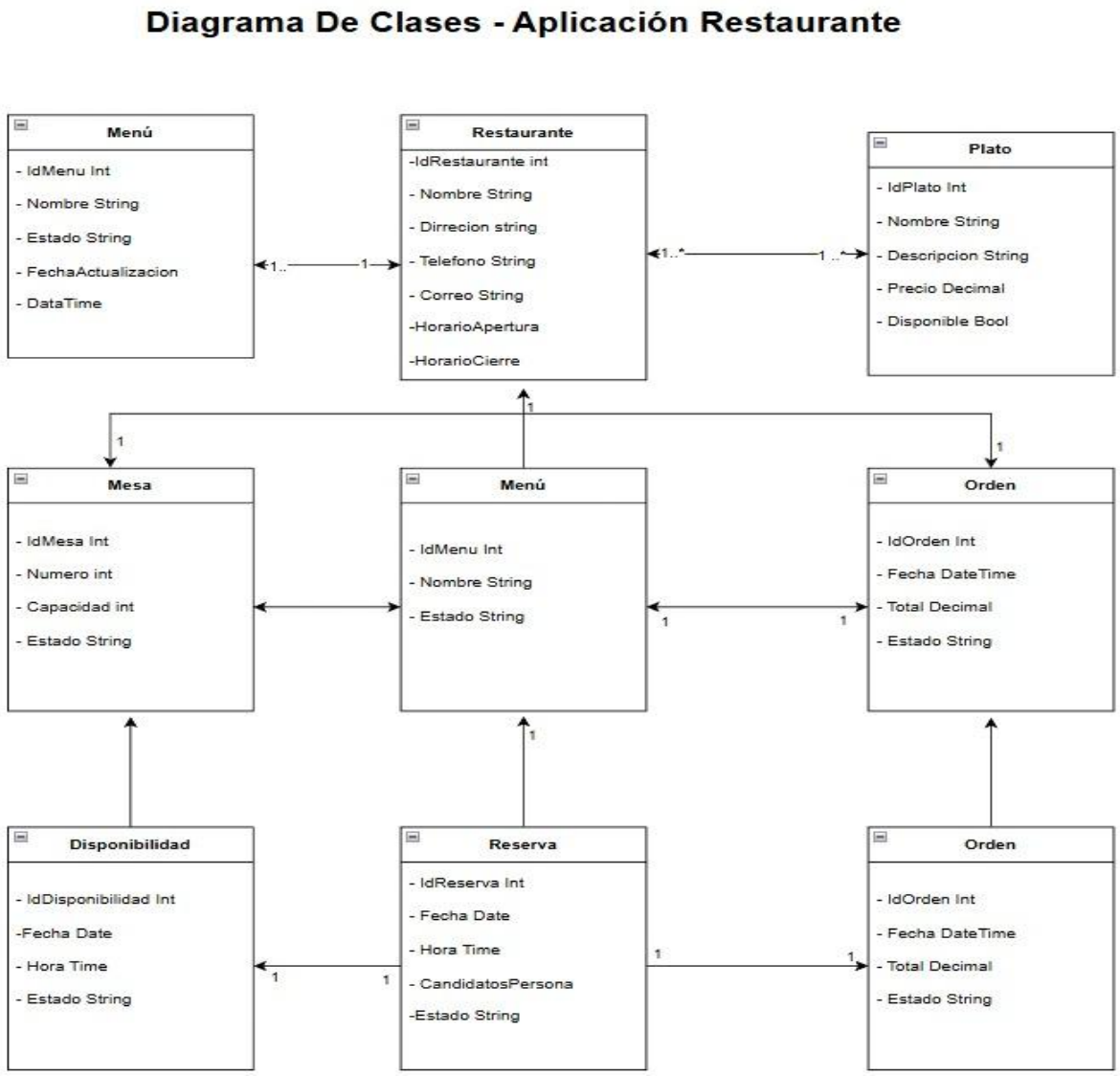
- **Disponibilidad**: Validada a través de las clases de Mesa y Disponibilidad para evitar choques de horarios.
- **Pagos**: Gestionados por la entidad de Pago que conecta la orden con la validación financiera.
- **Control del Servicio**: Administrado mediante clases de Auditoría y Moderación para asegurar la transparencia del sistema.

Aplicación Cliente



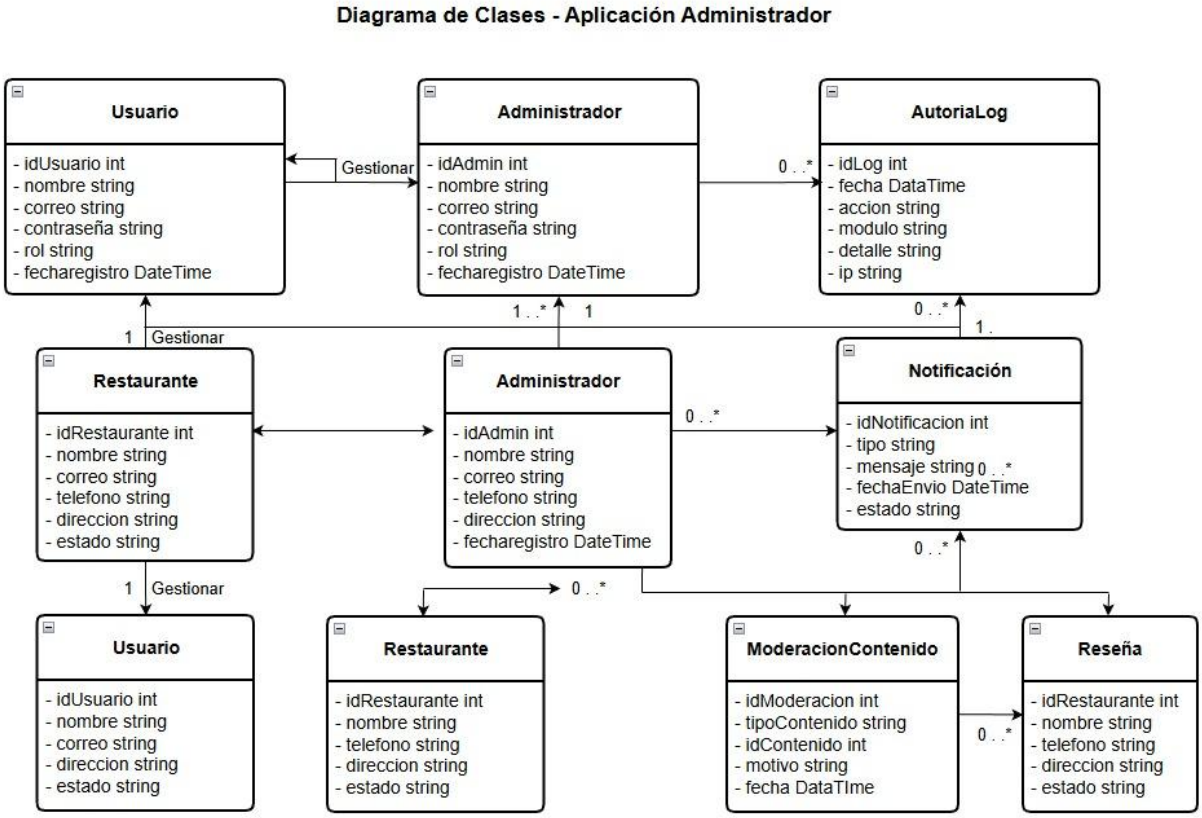
La Vista Lógica de nuestro sistema detalla la estructura interna de "Te Atiendo" mediante un estándar UML con un enfoque profundo a nivel de código, superando las descripciones generales de infraestructura al definir con precisión las entidades, atributos y relaciones que rigen el comportamiento del software. A través de la modularización del dominio en aplicaciones para el Cliente, Restaurante y Administrador, el diseño establece reglas de negocio estrictas donde cada clase —como Reserva, Orden o Pago— posee una estructura de datos normalizada que garantiza la integridad en SQL Server. Esta arquitectura permite que procesos complejos, como la validación de disponibilidad de mesas o el registro de AuditoríaLog, se ejecuten mediante servicios transversales y relaciones de mapeo bien definidas, asegurando que cada componente funcional del backend y frontend opere en total sincronía.

Aplicación Restaurante



Este último diagrama de clases de la Aplicación Restaurante funciona como el mapa que organiza toda la información interna del negocio en el código. Básicamente, conecta al Restaurante con sus Menús y Platos, permitiendo que el sistema controle qué hay disponible para comer y en qué Mesas se puede sentar la gente. Lo más importante es que amarra las Reservas de los clientes con la disponibilidad real del local y genera automáticamente una Orden para que la cocina sepa qué preparar, manteniendo todo sincronizado desde que el cliente elige su mesa hasta que se registra su pedido final.

Aplicación Administrador



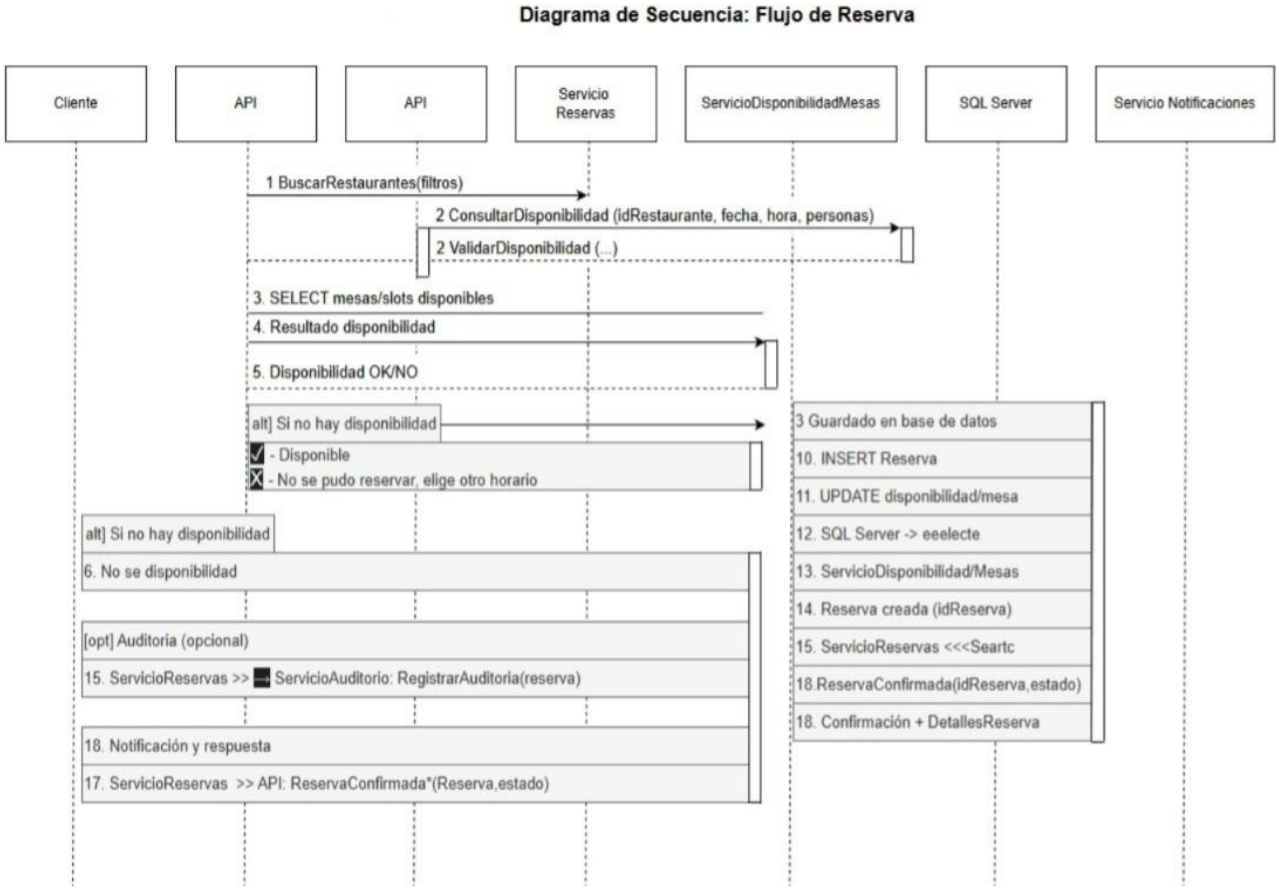
El diagrama de clases para la aplicación del administrador de **Te Atiendo** detalla la estructura lógica necesaria para centralizar el control operativo del ecosistema. La arquitectura se organiza en torno a la entidad **Administrador**, la cual tiene la responsabilidad de gestionar a los **Usuarios** y **Restaurantes**, además de supervisar la **Moderación de Contenido** y las **Reseñas** generadas en la plataforma. Para asegurar la transparencia y seguridad del sistema, el diseño incluye una clase de **AuditoriaLog** que registra detalles técnicos como la acción realizada y la dirección IP, junto con un sistema de **Notificaciones** para mantener la comunicación entre los distintos módulos.

La **Vista de Procesos** describe los flujos de ejecución del sistema "Te Atiendo" y su comportamiento dinámico durante las operaciones más críticas para el negocio. A través de diagramas de secuencia detallados a nivel de código, se ilustra cómo la API coordina la comunicación entre el frontend, los servicios de validación interna y las bases de datos para garantizar respuestas en tiempo real. Esta vista es fundamental para demostrar la robustez del software, asegurando que procesos complejos como la gestión de pagos y reservas sigan reglas de negocio estrictas antes de confirmar cualquier transacción.

Procesos relevantes:

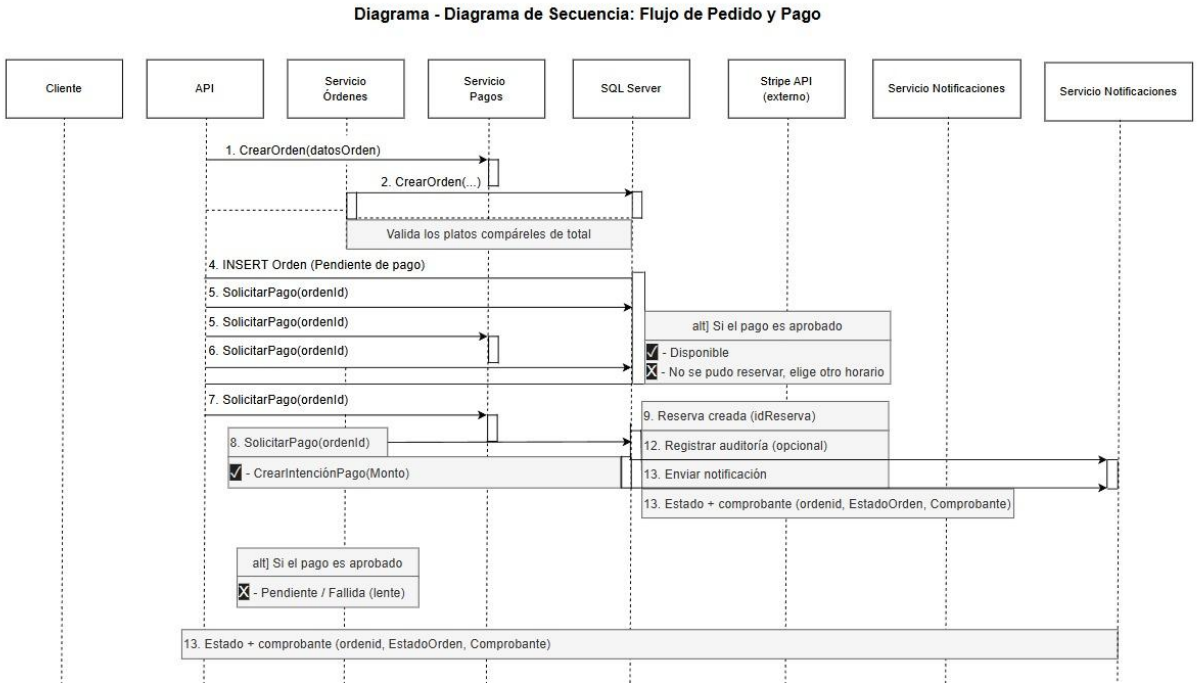
- **Flujo de reserva de mesa:** Describe la interacción desde la búsqueda de disponibilidad hasta el registro final en SQL Server, incluyendo la validación obligatoria de slots libres.
- **Registro de pedido y pago:** Detalla el ciclo de vida de una orden, integrando la verificación de precios y la autorización financiera segura a través de Stripe API.
- **Consumo y verificación de disponibilidad:** Proceso automático que bloquea recursos en tiempo real para evitar duplicidad de reservas o sobreventa de platos.
- **Registro de auditoría operativa:** Cada flujo crítico incluye un paso de registro en el servicio de auditoría, capturando eventos clave para el control administrativo y la seguridad del sistema.

Flujo de Reservas



El **Diagrama de Secuencia del Flujo de Reserva** para el sistema **Te Atiendo** ilustra la interacción cronológica entre el cliente y los componentes del backend, detallando cómo la **API** coordina con el **Servicio de Reservas** y el **Servicio de Disponibilidad de Mesas** para validar y procesar una solicitud. El flujo destaca la persistencia de datos en **SQL Server** mediante operaciones de inserción y actualización, condicionadas por la disponibilidad lógica del establecimiento, y concluye con el envío de confirmaciones a través del **Servicio de Notificaciones** y el registro de la operación en el **Servicio de Auditoría** para garantizar la trazabilidad del proceso.

Flujo De Pedido y pago



El **Diagrama de Secuencia de Flujo de Pedido y Pago** de la plataforma **Te Atiendo** describe el proceso transaccional donde la **API** coordina con el **Servicio de Órdenes** y el **Servicio de Pagos** para validar platos, calcular totales e insertar órdenes con estado pendiente en **SQL Server**. El flujo integra la pasarela externa **Stripe API** para crear la intención de cobro y, tras la aprobación, procesa la actualización de la reserva, el registro opcional en auditoría y el envío de notificaciones automáticas con el comprobante detallado al cliente.

Vista de desarrollo

Vista de Desarrollo: Te Atiendo

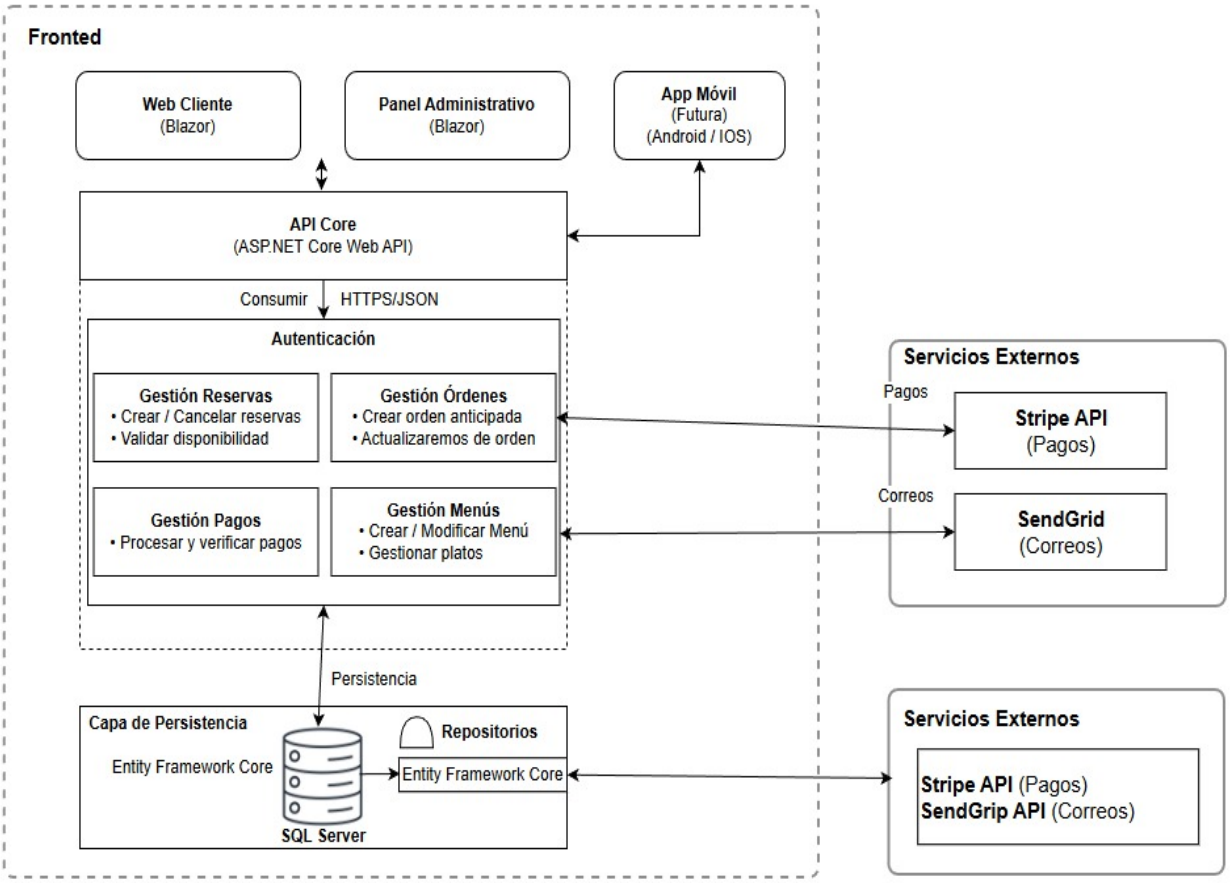
Esta vista describe la organización del software y las dependencias técnicas desde la perspectiva de la implementación.

Estructura de Módulos

El sistema se organiza en una solución de **.NET** compuesta por los siguientes proyectos:

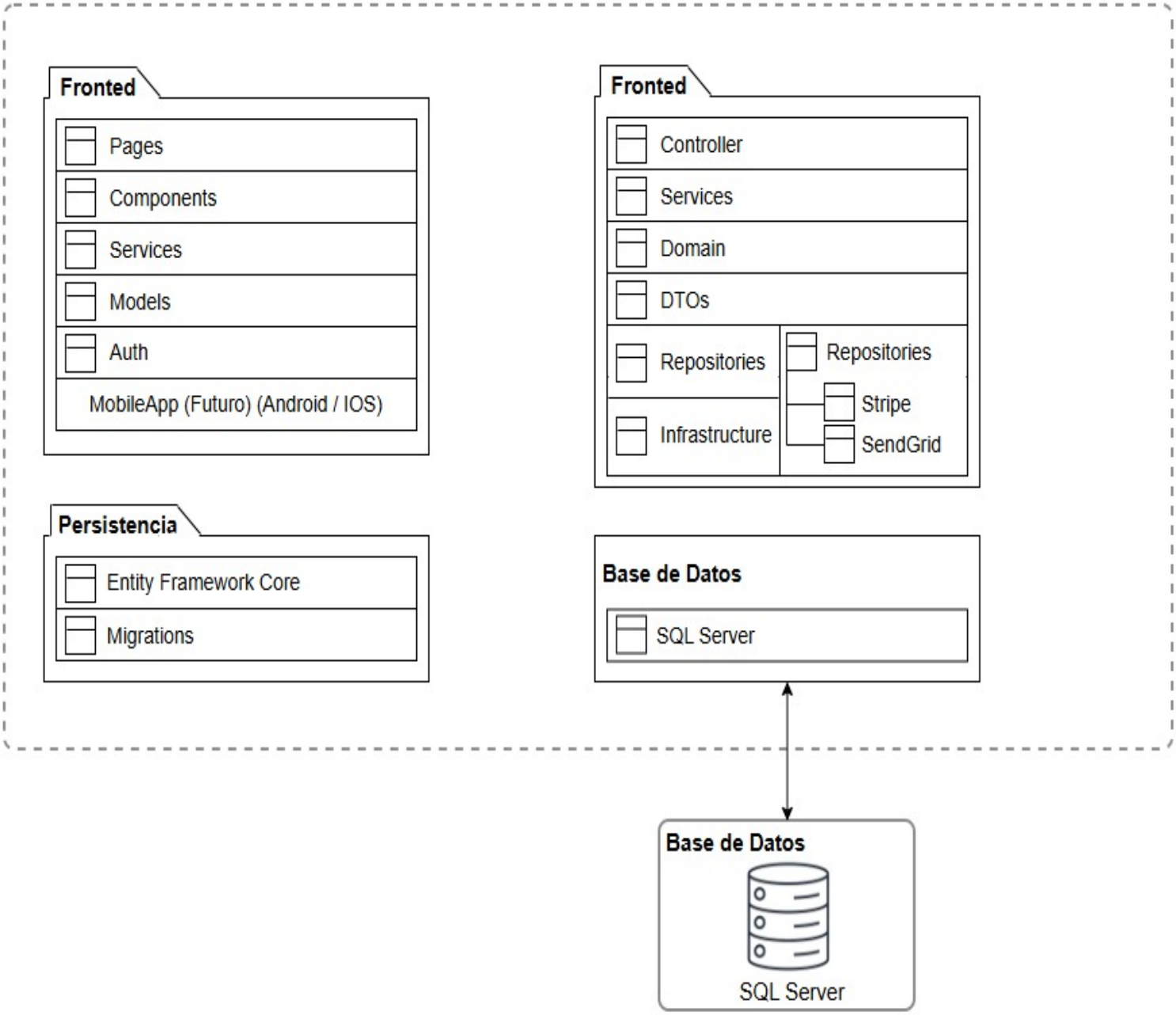
- **Frontend (Blazor):** Aplicaciones independientes para Cliente, Restaurante y Administrador.
- **Backend (API.Core):** Motor central en **ASP.NET Core** que expone servicios a través de **HTTPS/JSON**.
- **Infraestructura:** Implementaciones para la persistencia de datos y servicios externos (**Stripe**, **SendGrid**).
- **Dominio:** Biblioteca de clases con las entidades y reglas de negocio compartidas.

Diagrama componente



El diagrama de Componentes de Te Atiendo describe una arquitectura desacoplada en .NET, donde las interfaces en Blazor consumen una API Core central que orquesta la lógica de reservas, pagos y menús. Este diseño utiliza Entity Framework Core para la persistencia en SQL Server e integra servicios externos (Stripe/SendGrid), permitiendo la futura expansión a una App Móvil sin alterar el núcleo del sistema.

Diagrama de Paquetes --- Plataforma "Te Atiendo"



Vista de Despliegue

Diagrama de despliegue

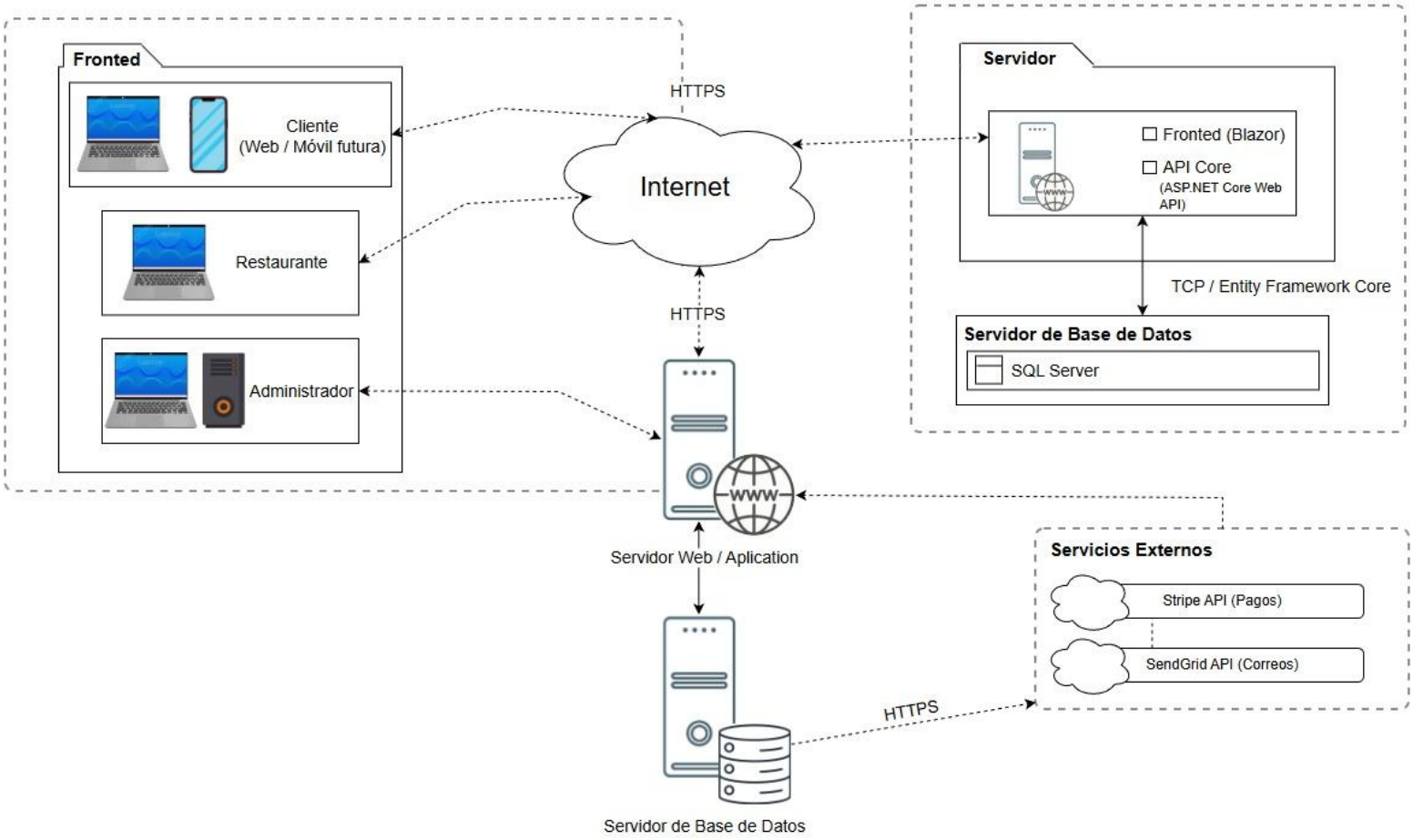
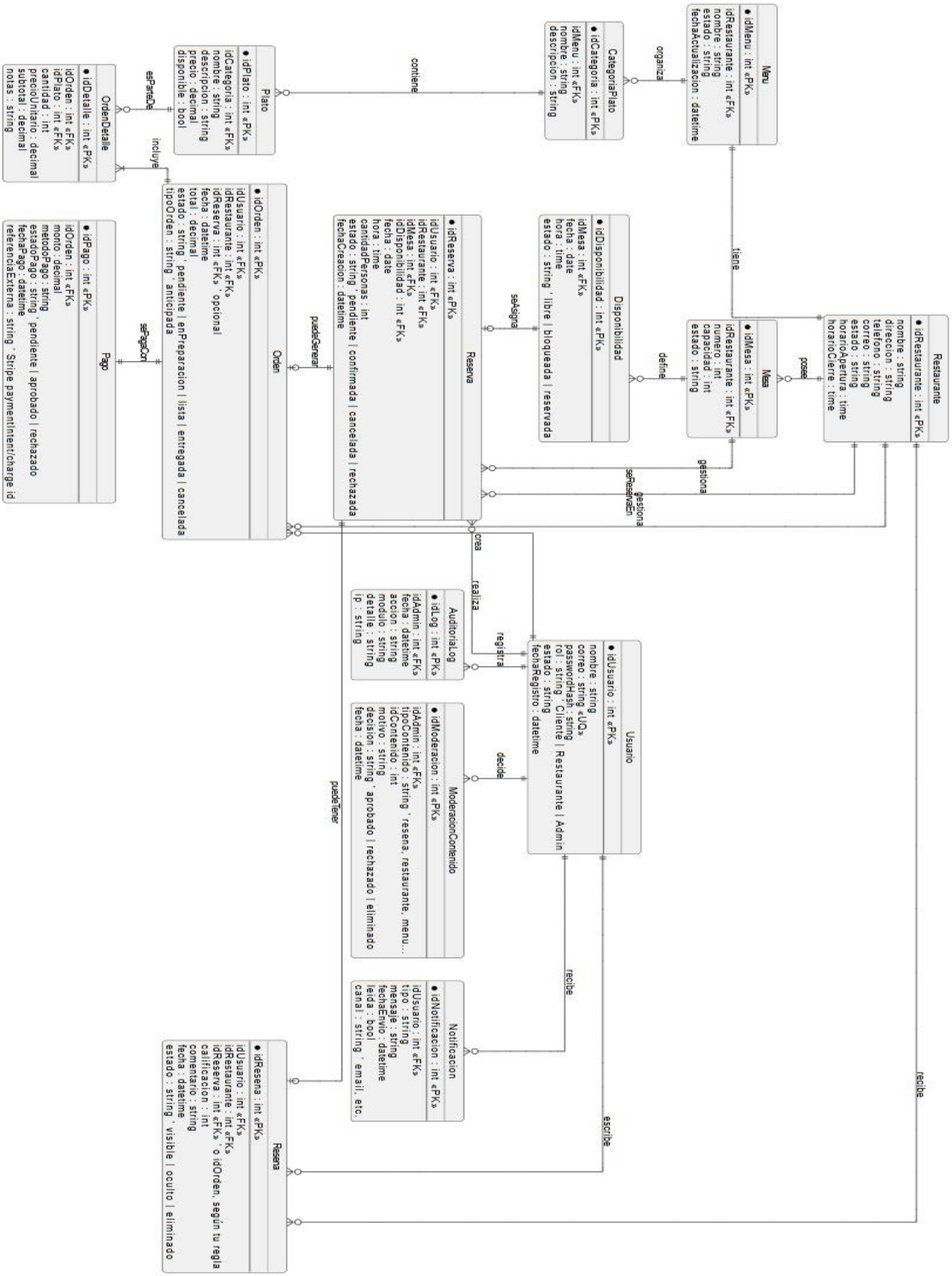


DIAGRAMA E-R BASE DE DATOS



Componentes del Sistema

El sistema "Te Atiendo" se fundamenta en un ecosistema de tecnologías modernas de Microsoft y servicios en la nube, diseñados para asegurar la escalabilidad, la seguridad transaccional y la facilidad de integración de sus módulos especializados.

Aplicación Web (Clientes)

Es la interfaz principal de interacción para los usuarios finales, desarrollada con el framework **Blazor**. A través de este canal, los clientes realizan el registro y autenticación, búsqueda de establecimientos, gestión de reservas en tiempo real, configuración de órdenes anticipadas y consulta de estados de pago y actividades históricas.

Aplicación Desktop (Administrador / Propietario)

Herramienta orientada a la gestión operativa de los locales y la supervisión global del sistema. Permite a los propietarios administrar menús, disponibilidad de mesas, procesamiento de pedidos y moderación de contenido, integrando además el control de auditoría y el envío de notificaciones administrativas.

API Backend (Servicio Central)

Constituye el núcleo lógico del ecosistema, implementado sobre **ASP.NET Core**. Este componente centraliza las reglas de negocio, el control de permisos, la validación de transacciones y la coordinación de servicios externos, exponiendo recursos mediante protocolos seguros **HTTPS** y formato de datos **JSON**.

Base de Datos

Componente encargado del almacenamiento persistente y relacional de toda la información del sistema. Utiliza **SQL Server** para gestionar de forma segura los perfiles de usuario, registros de transacciones financieras, disponibilidad de inventario físico y los logs de auditoría técnica.

Aplicación Móvil (Futura Integración)

Constituye un canal de acceso adicional diseñado para dispositivos iOS y Android que se integrará al ecosistema existente. Este componente permitirá a los usuarios realizar reservas y consultas mediante una interfaz nativa, aprovechando funciones del dispositivo como notificaciones push y geolocalización para mejorar la experiencia del cliente en movilidad.

Módulos del Sistema Te Atiendo

Aplicación Web (Cliente)

- Autenticación y gestión de usuarios
- Búsqueda y consulta de restaurantes
- Gestión de reservas
- Gestión de órdenes anticipadas
- Gestión de pagos
- Reseñas y evaluaciones
- Historial de actividades

Aplicación Desktop / Web (Administrador y Propietario)

- Gestión de restaurantes
- Gestión de menús

- Gestión de reservas
- Gestión de órdenes
- Moderación de reseñas
- Supervisión administrativa
- Auditoría de sistema
- Gestión de notificaciones

Decisiones Arquitectónicas

Para el desarrollo del sistema **Te Atiendo**, se tomaron decisiones arquitectónicas estratégicas con el objetivo de asegurar la integridad transaccional y la robustez del ecosistema gastronómico.

Una de las principales decisiones fue utilizar una **arquitectura de N-Capas**, estableciendo una diferenciación nítida entre la interacción del usuario, la orquestación de procesos y la infraestructura técnica. Esta estructura evita la mezcla de responsabilidades y garantiza que las reglas de negocio, como la validación de slots de mesas o el cálculo de órdenes, se concentren exclusivamente en el núcleo del sistema.

Asimismo, se decidió implementar una **API centralizada en ASP.NET Core**, la cual concentra la lógica de dominio y coordina los servicios de aplicación. Las aplicaciones desarrolladas en **Blazor** consumen esta API mediante comunicación segura **HTTPS/JSON**, asegurando que las reglas del restaurante se apliquen de manera uniforme en todos los puntos de acceso.

En cuanto a la trazabilidad, se optó por incluir **servicios transversales de Auditoría y Logging**. Esta decisión permite documentar cada operación administrativa y técnica de forma independiente a la lógica funcional, facilitando la supervisión institucional y el mantenimiento del código limpio.

La elección de una arquitectura de N-Capas facilita la evolución del sistema, permitiendo que cambios en la interfaz o en los proveedores de infraestructura no alteren las reglas de reserva o facturación. El diseño orientado al dominio asegura que el sistema sea coherente con las necesidades reales de un establecimiento gastronómico moderno.

El uso de **.NET 8** y **C#** se justifica por su capacidad para gestionar múltiples solicitudes simultáneas y su excelente integración con **Entity Framework Core**. El uso de repositorios permite separar el dominio del acceso técnico a los datos, mejorando la testabilidad y la productividad durante el desarrollo.

La centralización de servicios en el backend garantiza una seguridad transaccional superior, indispensable para manejar información delicada como el historial de pagos y la disponibilidad física de mesas. Esta estructura facilita la incorporación futura de una **aplicación móvil**, ya que esta consumirá los mismos recursos y validaciones ya establecidos en el núcleo central.

Reglas de Negocio ↔ Arquitectura

La arquitectura integra las reglas operativas del restaurante directamente en su flujo dinámico, asegurando que cada interacción sea válida y controlada.

- **Validación Centralizada:** La capa de dominio valida en tiempo real la **disponibilidad de mesas**, asegurando que solo se concreten reservas en horarios y capacidades físicas reales del local
- **Control Transaccional de Pedidos:** Los menús y órdenes se manejan como elementos del dominio que determinan la vigencia de los platos y sus precios actuales, evitando discrepancias entre lo visualizado y lo facturado.
- **Supervisión y Auditoría:** La arquitectura implementa de forma transversal el registro de **AuditoriaLog**, capturando cada acción relevante para garantizar la trazabilidad y la vigilancia operativa por parte del administrador.

Requisitos No Funcionales

- **Seguridad:** El acceso se restringe mediante roles (Cliente, Restaurante, Admin), asegurando que cada perfil interactúe solo con sus funciones permitidas y que toda acción administrativa sea rastreada por IP y fecha.
- **Disponibilidad:** El sistema debe operar de forma continua para permitir reservas externas en cualquier horario; su arquitectura centralizada facilita la recuperación ante incidentes y reduce los puntos únicos de fallo.

- **Rendimiento:** Las operaciones críticas, como el procesamiento de pagos en Stripe y la validación de disponibilidad, deben ejecutarse en tiempos mínimos para evitar la pérdida de transacciones durante picos de demanda.
- **Trazabilidad:** Se garantiza que cada movimiento de dinero o cambio en el estado de una reserva quede registrado, permitiendo determinar quién, cuándo y qué operación se realizó en el sistema.

Riesgos Técnicos

- **Sobrecarga en Horarios Pico:** El alto volumen de reservas simultáneas puede estresar la base de datos. **Mitigación:** Una arquitectura dividida que permite optimizar las consultas críticas y el uso de transacciones atómicas en SQL Server.
- **Fallos en Servicios Externos:** La dependencia de Stripe o SendGrid podría afectar el cobro o la notificación. **Mitigación:** Manejo controlado de excepciones y almacenamiento temporal de estados para reintentar operaciones una vez se restablezca la conectividad externa.
- **Inconsistencia de Datos:** Errores en la red podrían generar órdenes pagadas pero no registradas. **Mitigación:** Implementación de flujos de verificación cruzada con la API de pagos para confirmar estados antes de actualizar el sistema local.

Conclusiones

El proyecto **Te Atiendo** ofrece una solución integral para la automatización de reservas, gestión de pedidos y procesamiento de pagos en la industria gastronómica, resolviendo de manera efectiva los desafíos operativos detectados inicialmente. Basándose en los requerimientos del SRS, se ha establecido una estructura técnica clara que permite comprender el ecosistema desde múltiples dimensiones.

La implementación del modelo de vistas arquitectónicas fue fundamental para organizar el diseño, permitiendo una separación nítida de responsabilidades, la identificación de los actores clave y la validación de los procesos críticos del sistema. Esto asegura que funciones vitales como la gestión de mesas, el control de menús y la seguridad transaccional estén alineadas estrictamente con las reglas de negocio de la plataforma.

Asimismo, la integración de los requisitos no funcionales ha guiado las decisiones sobre seguridad, trazabilidad y escalabilidad, garantizando un entorno confiable mediante el uso de tecnologías como **ASP.NET Core** y **SQL Server**. La identificación temprana de riesgos técnicos y la planificación de sus mitigaciones refuerzan la viabilidad del proyecto, reduciendo significativamente la posibilidad de fallos durante la fase de implementación.

En conclusión, la arquitectura propuesta para **Te Atiendo** constituye una base sólida y escalable que garantiza un manejo centralizado y eficaz de las operaciones del restaurante. Esta estructura no solo facilita el mantenimiento a largo plazo, sino que también brinda una experiencia uniforme y optimizada tanto para los comensales como para los propietarios y administradores del sistema.

.